Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет военный

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»
на тему

ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО

Вариант 12

Курсант гр. 434701 А.А. Медведь

Руководитель О.М. Внук

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы:

– изучить методы работы с интегрированной средой разработки Microsoft Visual Studio Code при использовании объектно-ориентированного языка программирования C++;

– научиться разрабатывать в выбранной среде приложение с графическим интерфейсом;

– изучить MVC с активной и пассивной моделью.

Задачи работы:

– выбрать объектно-ориентированный язык программирования и среду разработки;

– установить и настроить программное обеспечение, ознакомиться с его функциональными возможностями и средствами отладки, привести простые примеры работы с функционалом и продемонстрировать работу отладчика;

– создать пустой проект под выданный вариант задания, установить расширения для работы с SysML v2 и UML;

– разработать каркас MVC с выданной по варианту моделью.

Задание по варианту 12: разработать приложение «Астролог». На главном окне должна быть кнопка «Узнать свой знак Зодиака». По нажатию на кнопку открывается дополнительная форма для ввода даты рождения; стандартный элемент управления для ввода даты не использовать. Последние введенные данные нужно запоминать в главной форме и восстанавливать при повторном вводе. После этого на главной форме отображается вычисленный знак Зодиака и астрологическая информация про этот знак. Если введены некорректные данные – показать сообщение об ошибке. Добавить каркас MVC с активной моделью.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Интегрированная среда разработки (IDE, Integrated Development Environment) – комплекс программных средств, объединяющий в одном приложении инструменты, необходимые на всех этапах создания программы. В состав IDE обычно входят:

– редактор исходного кода с подсветкой синтаксиса, автодополнением и навигацией по коду;

– компилятор или интерпретатор языка программирования;

– средства автоматизации сборки;

– отладчик;

– средства управления проектом и интеграции с системой управления версиями;

– система расширений, позволяющая добавлять поддержку новых языков и инструментов.

Microsoft Visual Studio Code – бесплатный кроссплатформенный редактор кода с функциями интегрированной среды разработки. Поддержка конкретного языка подключается расширениями из магазина Visual Studio Marketplace. Для языка C++ используется расширение C/C++ компании Microsoft, которое добавляет анализ кода IntelliSense и интеграцию с отладчиками GDB, LLDB и Microsoft C++. Сборка выполняется задачами (файл tasks.json), запуск под отладчиком – по конфигурациям запуска (файл launch.json) [1].

Отладчик позволяет выполнять программу под контролем: останавливать ее в точках останова, выполнять пошагово, просматривать значения переменных и стек вызовов, вычислять выражения в контексте остановки. Отладчик GDB (GNU Debugger) входит в состав набора компиляторов GCC и используется средой через протокол Machine Interface.

UML (Unified Modeling Language) – унифицированный язык графического описания для объектного моделирования программных систем; диаграмма классов показывает классы системы, их атрибуты, методы и связи между ними. SysML v2 (Systems Modeling Language) – язык моделирования систем, вторая версия которого имеет текстовую нотацию: части системы (part def), атрибуты (attribute def), порты (port def), действия (action def) и требования (requirement def) описываются в текстовом файле [2].

MVC (Model – View – Controller) – архитектурный шаблон, разделяющий приложение на три компонента:

– модель (Model) – данные и бизнес-логика, не зависящие от способа отображения;

– представление (View) – отображение данных модели пользователю;

– контроллер (Controller) – обработка действий пользователя и изменение модели.

Различают два варианта шаблона. Пассивная модель не знает о представлениях: после ее изменения контроллер сам передает новые данные представлению. Активная модель самостоятельно оповещает подписанные на нее представления об изменении своего состояния, как правило с помощью шаблона «Наблюдатель» (Observer): представление регистрируется в модели как слушатель, а модель при каждом изменении вызывает у всех слушателей метод обновления [3].

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Выбор языка программирования и среды разработки

Для выполнения работы выбран объектно-ориентированный язык C++ (стандарт C++17) [4] и среда разработки Microsoft Visual Studio Code 1.141.2. Компиляция выполняется компилятором GCC 16.2.0 из сборки MinGW-w64 (WinLibs), отладка – отладчиком GDB 18.1. Графический интерфейс реализован средствами Windows API [5], которые входят в состав операционной системы и не требуют подключения сторонних библиотек. Состав использованного программного обеспечения приведен в таблице 3.1.

Таблица 3.1 – Состав программного обеспечения

Программное средство	Версия	Назначение
Microsoft Visual Studio Code	1.141.2	Редактирование, сборка, отладка
Расширение C/C++	1.34.4	Анализ кода C++, интеграция с GDB
Компилятор GCC (MinGW-w64)	16.2.0	Компиляция программы на C++
Отладчик GDB	18.1	Отладка программы
Расширение PlantUML	2.18.1	Построение диаграмм UML
Расширение Syside Editor	0.11.0	Поддержка языка SysML v2

Из таблицы 3.1 видно, что все средства разработки, кроме самой среды, подключаются к Visual Studio Code расширениями или внешними программами: компилятор и отладчик – через расширение C/C++, языки моделирования – через расширения PlantUML и Syside Editor.

3.2 Установка и настройка программного обеспечения

Среда Visual Studio Code и компилятор GCC установлены с официальных источников. Каталог bin компилятора добавлен в переменную среды PATH, что проверено командами g++ --version и gdb --version во встроенном терминале среды. В среде установлено расширение C/C++ (ms-vscode.cpptools).

Для сборки в каталоге .vscode проекта создан файл tasks.json с задачей «Сборка (g++)», которая компилирует исходные файлы с ключами -std=c++17 (стандарт языка), -g (отладочная информация), -municode и -mwindows (оконное приложение Windows с точкой входа wWinMain). Для отладки создан файл launch.json с конфигурацией «Отладка (gdb)», которая

перед запуском выполняет сборку. Главное окно среды с открытым проектом приведено на рисунке 3.1.

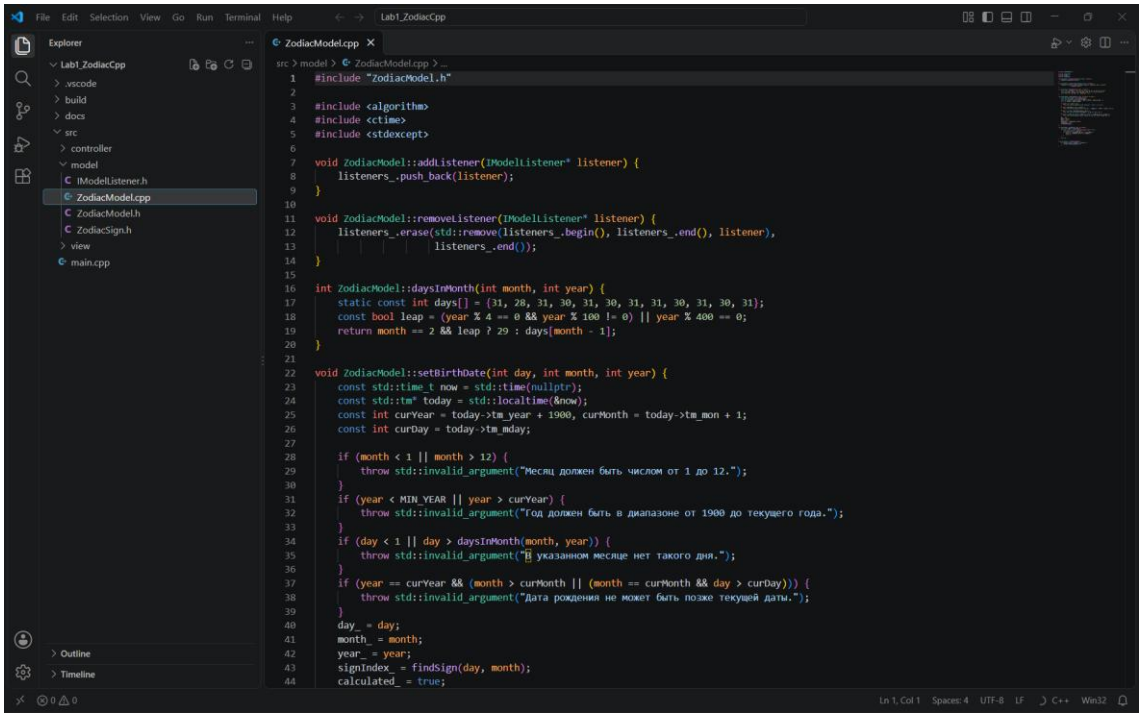


Рисунок 3.1 – Главное окно Visual Studio Code с открытым проектом

Примеры работы с функционалом среды:

– сборка проекта сочетанием клавиш Ctrl+Shift+B, результат которой выводится во встроенный терминал (рисунок 3.2);

– переход к определению функции (F12) и поиск всех ссылок (Shift+F12);

– автодополнение IntelliSense (Ctrl+Space) и подсказка параметров функций Windows API;

– быстрое открытие файла по имени (Ctrl+P);

– переименование идентификатора во всех файлах проекта (F2).



Рисунок 3.2 – Результат сборки проекта

Отладчик запускается клавишей F5. Для демонстрации его работы точка останова установлена в методе ZodiacController::onInputData() на строке передачи даты модели. После ввода даты рождения 15.03.2005 выполнение программы остановилось на этой строке (рисунок 3.3). На панели Variables отображаются значения локальных переменных day = 15, month = 3, year = 2005, полученных из текстовых полей окна ввода, на панели Call Stack – цепочка вызовов от оконной процедуры MainWindow::windowProc() до обработчика кнопки.

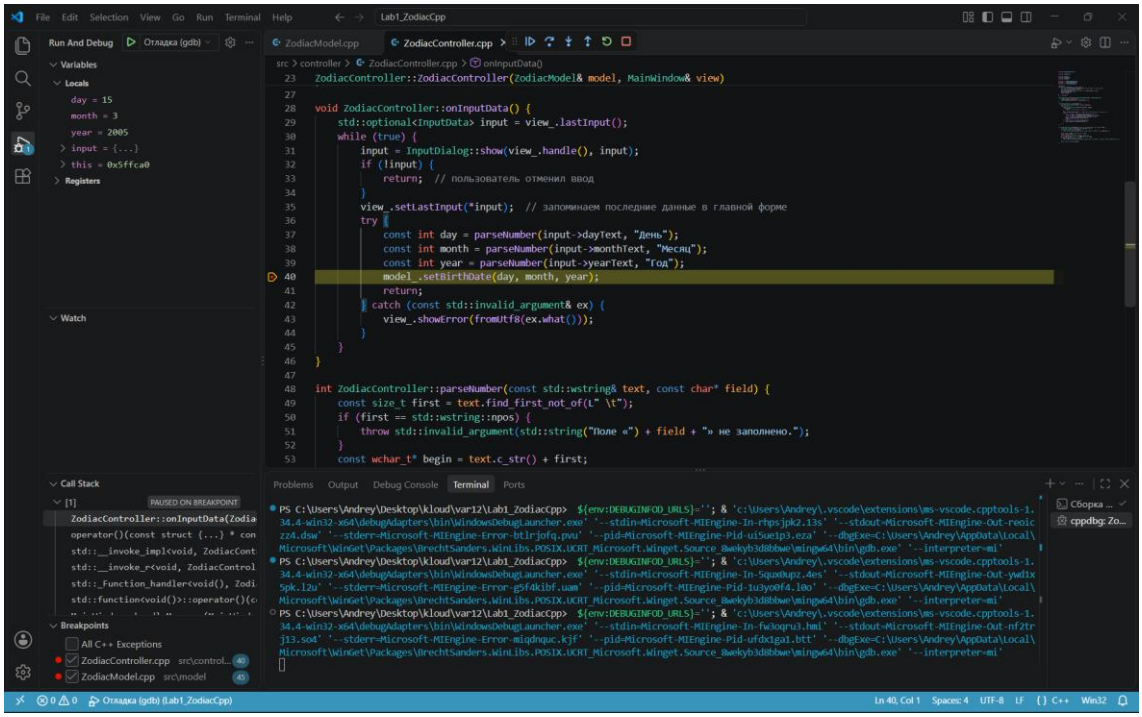


Рисунок 3.3 – Остановка в методе ZodiacController::onInputData()

Вторая точка останова установлена в методе ZodiacModel::setBirthDate() на строке оповещения слушателей. После продолжения выполнения (F5) программа остановилась на ней. На панели Variables раскрыт объект this: модель сохранила дату и определила индекс знака signIndex_ = 2, что соответствует знаку «Рыбы». В окне Debug Console вычислено выражение zodiacSigns()[2].startDay, равное 19, – начало периода знака (рисунок 3.4). Пошаговое выполнение выполняется клавишами F10 (шаг с обходом) и F11 (шаг с заходом).

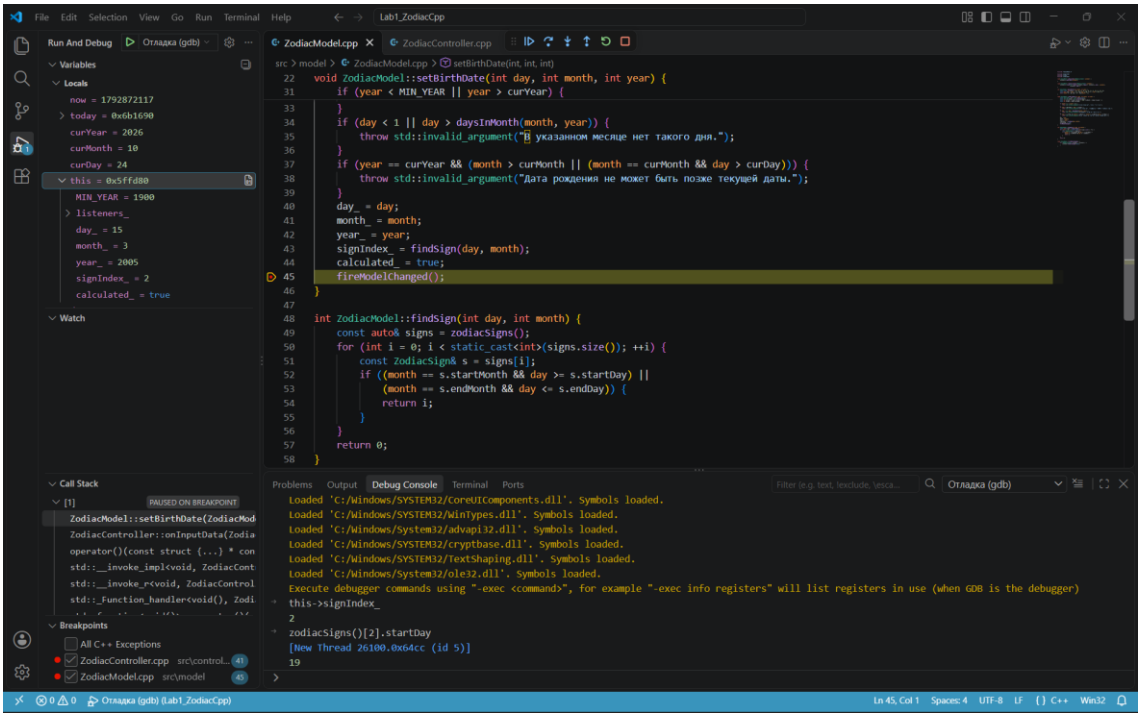


Рисунок 3.4 – Остановка в методе ZodiacModel::setBirthDate()

3.3 Создание проекта и установка расширений для SysML v2 и UML

Проект создан как каталог Lab1_ZodiacCpp, открытый в среде командой File → Open Folder. Исходные тексты размещены в каталоге src и разделены по компонентам шаблона MVC: model, view и controller. Диаграммы и модель системы размещены в каталоге docs.

Через панель Extensions (Ctrl+Shift+X) установлены расширения, предложенные в задании для Visual Studio Code: PlantUML (jebbs.plantuml) для работы с UML и Syside Editor: SysML v2 Essential (sensmetry.syside-editor) для работы с SysML v2. Расширение PlantUML добавляет подсветку синтаксиса файлов .puml и окно предварительного просмотра диаграмм (Alt+D), расширение Syside Editor – подсветку синтаксиса и проверку файлов .sysml. Список установленных расширений приведен на рисунке 3.5.

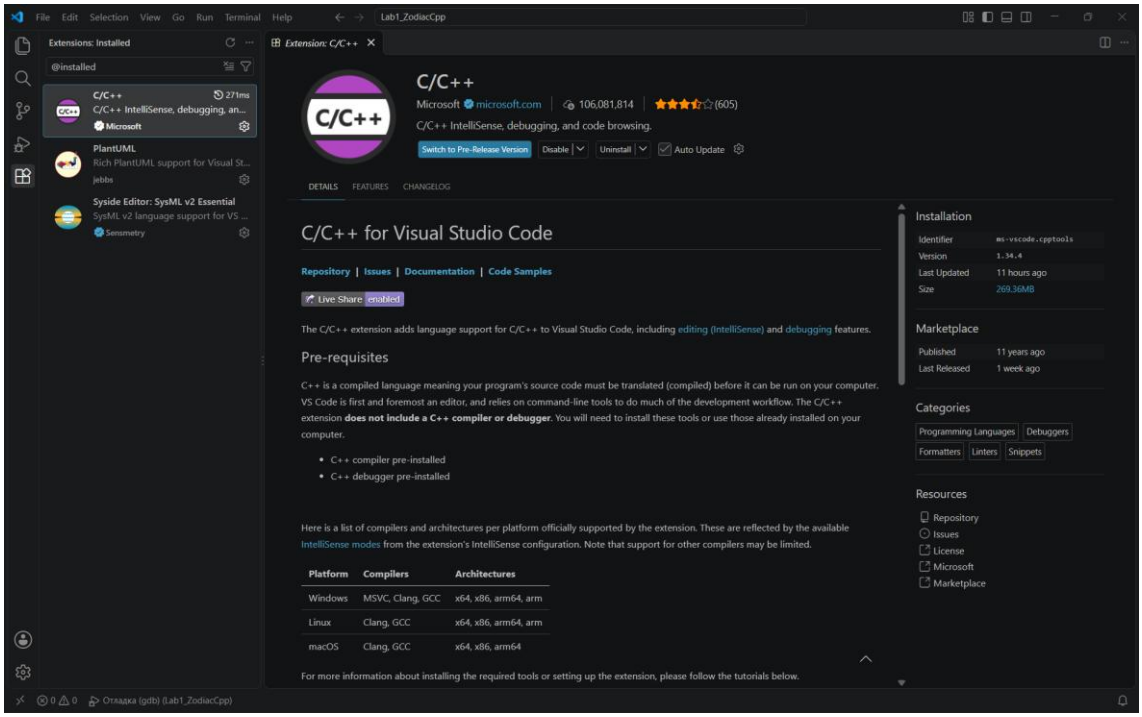


Рисунок 3.5 – Установленные расширения Visual Studio Code

Диаграмма классов приложения описана в нотации PlantUML в файле docs/class-diagram.puml и отображается расширением в окне предварительного просмотра (рисунок 3.6).

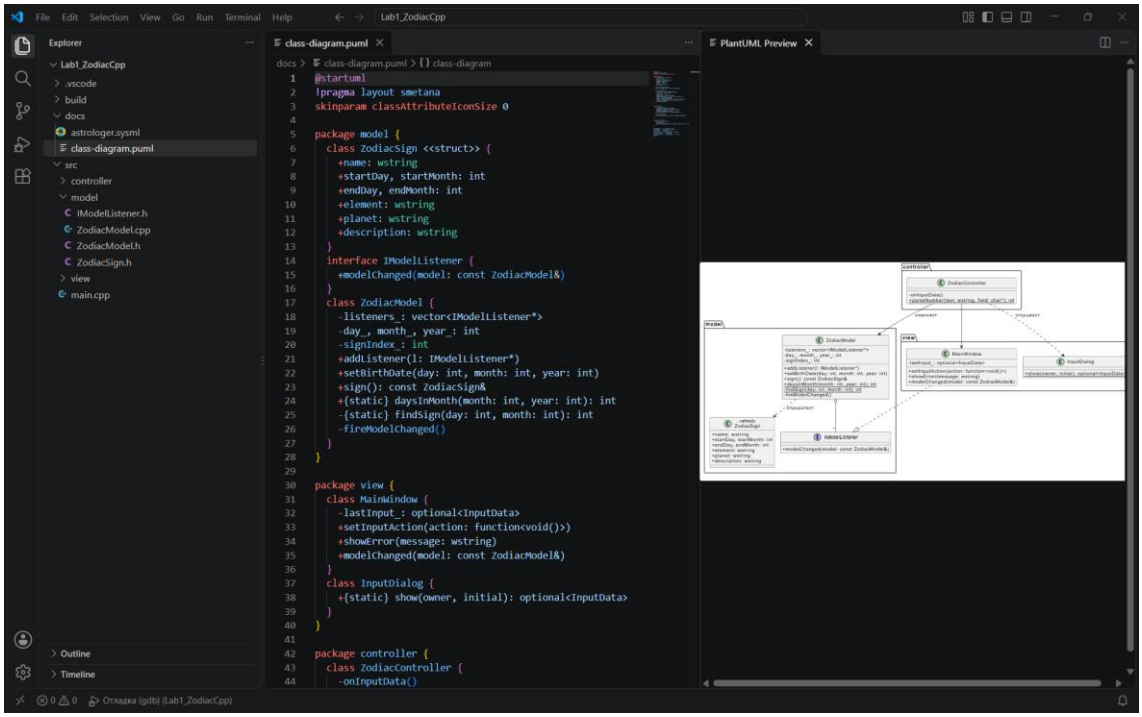


Рисунок 3.6 – Диаграмма классов в окне предварительного просмотра PlantUML

Модель системы в нотации SysML v2 описана в файле docs/astrologer.sysml: определены данные окна ввода и знака Зодиака, части системы zodiacModel, mainWindow, inputDialog и zodiacController, порт уведомления модели и действие определения знака Зодиака. Файл в редакторе с подсветкой синтаксиса расширения приведен на рисунке 3.7.

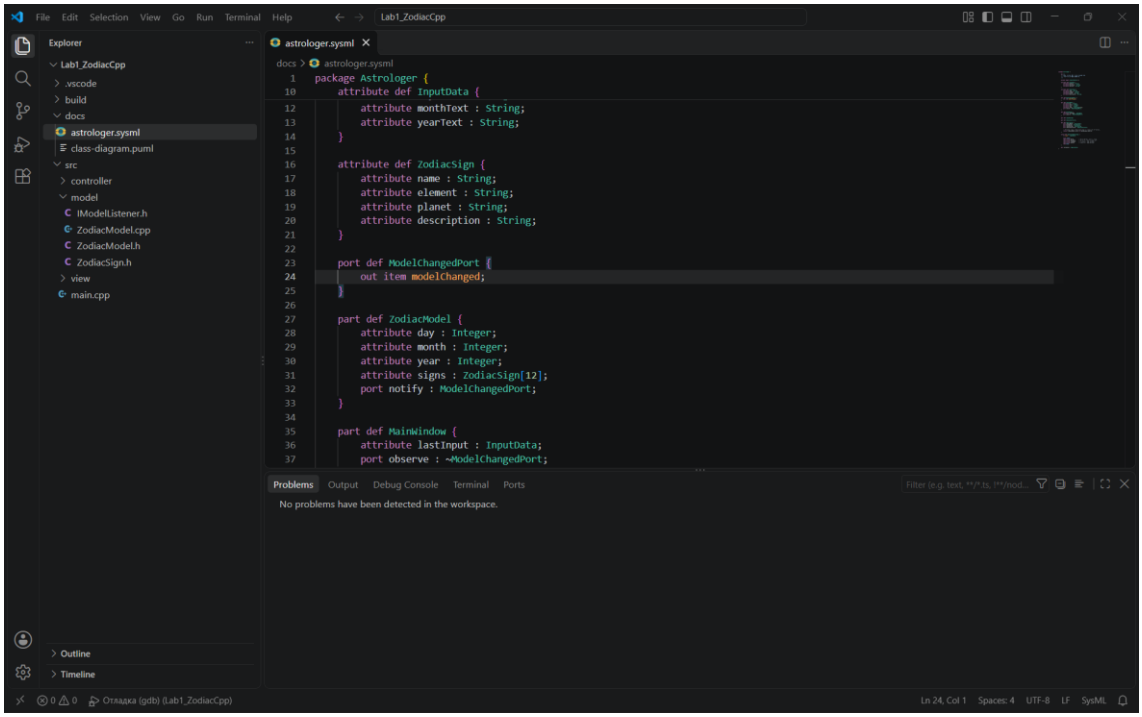


Рисунок 3.7 – Модель системы в нотации SysML v2

3.4 Каркас MVC с активной моделью

Приложение построено по шаблону MVC с активной моделью. Классы приложения и их связи приведены на диаграмме классов (см. рисунок 3.6). Назначение классов приведено в таблице 3.2.

Таблица 3.2 – Классы приложения

Класс	Компонент MVC	Назначение
ZodiacModel	Модель	Проверка даты, определение знака, оповещение слушателей
IModelListener	Модель	Интерфейс наблюдателя модели
ZodiacSign	Модель	Знак Зодиака и астрологическая информация
MainWindow	Представление	Главная форма, отображение знака Зодиака
InputDialog	Представление	Модальное окно ввода даты рождения
ZodiacController	Контроллер	Обработка кнопки «Узнать свой знак Зодиака»

Из таблицы 3.2 видно, что классы модели не используют функции Windows API: работа с окнами сосредоточена в классах представления, а контроллер связывает их с моделью.

Модель ZodiacModel не зависит от классов графического интерфейса. Метод setBirthDate() проверяет, что номер месяца положителен и не больше 12, год – от 1900 до текущего года, день существует в указанном месяце с учетом високосного года, а дата не позже текущей. Затем модель определяет знак Зодиака по дню и месяцу рождения и вызывает метод fireModelChanged(), который оповещает всех слушателей через интерфейс IModelListener. Периоды знаков Зодиака и сведения о них приведены в таблице 3.3.

Таблица 3.3 – Знаки Зодиака

Знак	Период	Стихия	Планета
Козерог	22.12–19.01	Земля	Сатурн
Водолей	20.01–18.02	Воздух	Уран
Рыбы	19.02–20.03	Вода	Нептун
Овен	21.03–19.04	Огонь	Марс
Телец	20.04–20.05	Земля	Венера
Близнецы	21.05–20.06	Воздух	Меркурий
Рак	21.06–22.07	Вода	Луна
Лев	23.07–22.08	Огонь	Солнце
Дева	23.08–22.09	Земля	Меркурий
Весы	23.09–22.10	Воздух	Венера
Скорпион	23.10–21.11	Вода	Плутон
Стрелец	22.11–21.12	Огонь	Юпитер

Из таблицы 3.3 видно, что периоды знаков следуют друг за другом без пропусков, поэтому любой дате года соответствует ровно один знак. Знаки хранятся в массиве структур ZodiacSign вместе с характеристикой, которая выводится на главной форме.

Представление MainWindow реализует интерфейс IModelListener и подписывается на модель в функции wWinMain() вызовом model.addListener(&view). В методе modelChanged() главная форма сама считывает из модели дату и знак Зодиака. Последние введенные данные хранятся в главной форме в поле lastInput_ и передаются в окно ввода InputDialog при повторном открытии. Дата рождения вводится в три текстовых поля – день, месяц и год, стандартный элемент выбора даты не используется.

Контроллер ZodiacController назначает обработчик кнопки «Узнать свой знак Зодиака». Обработчик открывает окно ввода, сохраняет введенные

данные в главной форме, преобразует текст полей в целые числа и передает дату модели. При ошибке преобразования или недопустимой дате контроллер выводит сообщение об ошибке и снова открывает окно ввода. Обновлением главной формы контроллер не занимается – это выполняет активная модель.

При запуске программы отображается главная форма с кнопкой «Узнать свой знак Зодиака» (рисунок 3.8). По нажатию кнопки открывается дополнительная форма для ввода дня, месяца и года рождения (рисунок 3.9).

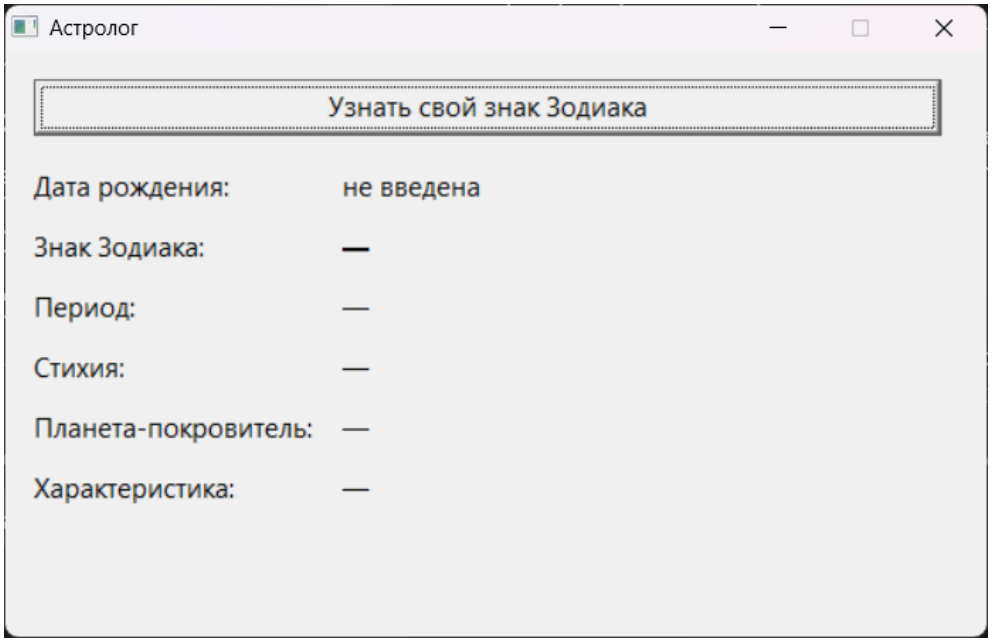


Рисунок 3.8 – Главная форма при запуске программы

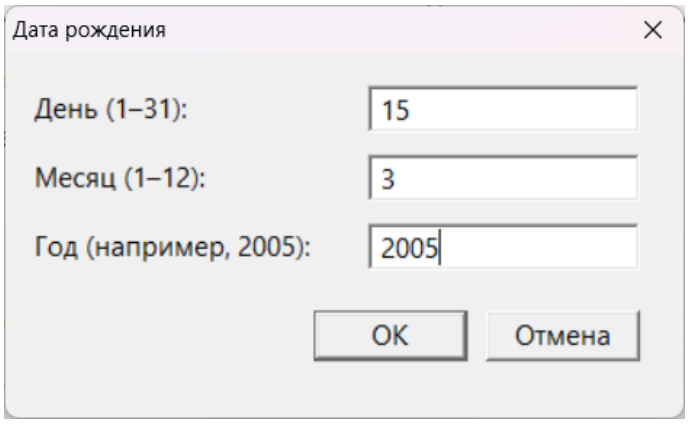


Рисунок 3.9 – Форма ввода даты рождения

После ввода даты 15.03.2005 и нажатия кнопки «OK» модель определяет знак Зодиака и оповещает главную форму, которая отображает

знак «Рыбы», его период с 19 февраля по 20 марта, стихию, планету-покровителя и характеристику (рисунок 3.10).

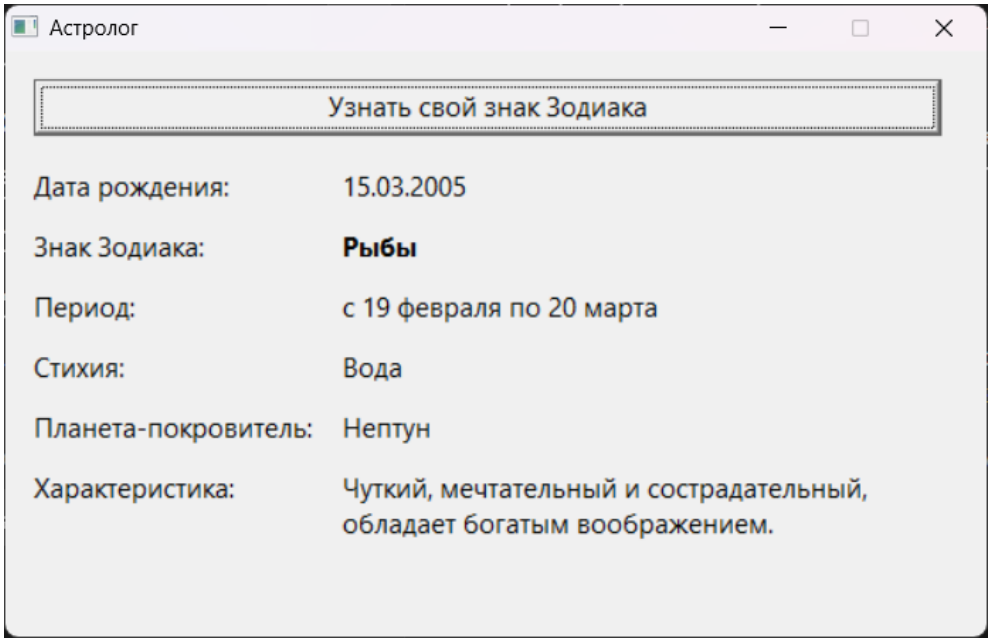


Рисунок 3.10 – Главная форма с вычисленным знаком Зодиака

При повторном нажатии кнопки в форме ввода восстанавливаются последние введенные данные (рисунок 3.11). При вводе в поле текста, не являющегося целым числом, выводится сообщение об ошибке формата (рисунок 3.12), а при вводе несуществующей даты, например 31.02.2005, – сообщение, сформированное моделью (рисунок 3.13).

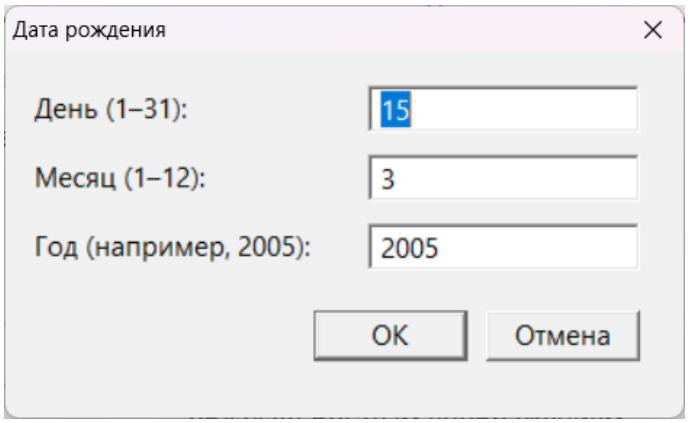


Рисунок 3.11 – Восстановление последних введенных данных

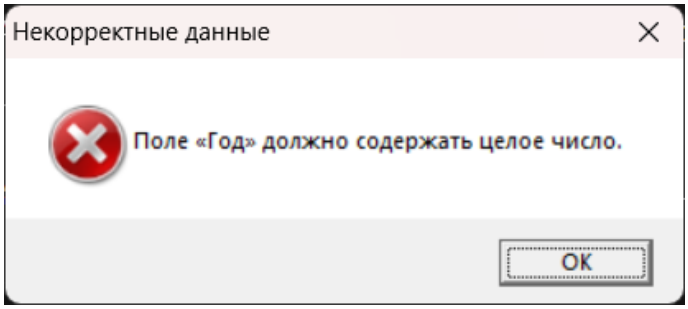


Рисунок 3.12 – Сообщение об ошибке при нечисловом вводе

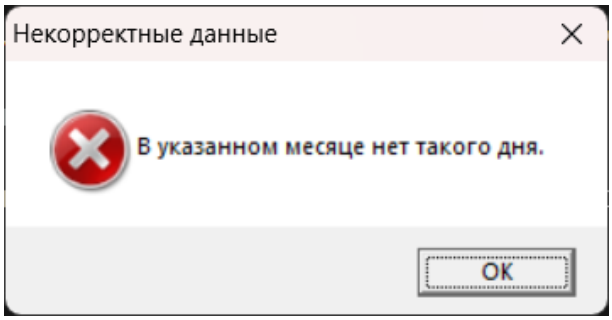


Рисунок 3.13 – Сообщение об ошибке при несуществующей дате

Исходные тексты программы приведены в приложении А.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор исходного кода с подсветкой синтаксиса и автодополнением, компилятор или интерпретатор языка, средства автоматизации сборки, отладчик, средства управления проектом, интеграция с системой управления версиями, встроенный терминал и система расширений. В Visual Studio Code компилятор и отладчик подключаются внешними инструментами (GCC, GDB) через расширение C/C++.

2 Каковы основные приемы работы с редактором кода?

Основные приемы: автодополнение (Ctrl+Space), переход к определению (F12) и поиск ссылок (Shift+F12), быстрое открытие файла (Ctrl+P), палитра команд (Ctrl+Shift+P), переименование идентификатора (F2), форматирование документа (Shift+Alt+F), комментирование строк (Ctrl+/), множественные курсоры (Alt+щелчок), сворачивание блоков кода.

3 Каким образом выполняется сборка и запуск проекта?

Сборка выполняется задачей из файла .vscode/tasks.json (Ctrl+Shift+B): среда вызывает компилятор g++, который компилирует исходные файлы и компонует исполняемый файл build/Zodiac.exe. Запуск под отладчиком выполняется клавишей F5 по конфигурации из файла .vscode/launch.json, запуск без отладки – сочетанием Ctrl+F5; перед запуском автоматически выполняется сборка.

4 Что такое MVC?

MVC (Model – View – Controller) – архитектурный шаблон, разделяющий приложение на модель (данные и бизнес-логика), представление (отображение данных) и контроллер (обработка действий пользователя). Разделение уменьшает связанность компонентов, позволяет изменять интерфейс без изменения логики и иметь несколько представлений одной модели.

5 Чем отличается активная модель от пассивной?

Пассивная модель не знает о представлениях: после ее изменения контроллер сам обновляет представление. Активная модель самостоятельно оповещает подписанные на нее представления об изменении состояния по шаблону «Наблюдатель», а контроллер только передает ей входные данные.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены методы работы с интегрированной средой разработки Microsoft Visual Studio Code при использовании языка C++: установка и настройка компилятора GCC и отладчика GDB, настройка задач сборки и конфигураций запуска, основные приемы работы с редактором кода. Изучены средства отладки: установка точек останова, просмотр значений переменных и стека вызовов, вычисление выражений, пошаговое выполнение.

Установлены расширения для работы с UML и SysML v2, построены диаграмма классов приложения и модель системы в текстовой нотации SysML v2.

Изучен шаблон MVC с активной и пассивной моделью. Разработано приложение с графическим интерфейсом «Астролог» на каркасе MVC с активной моделью: модель самостоятельно проверяет дату рождения, определяет знак Зодиака и оповещает главную форму, а контроллер только передает модели введенные пользователем данные.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Visual Studio Code. Documentation [Электронный ресурс]. – Режим доступа : https://code.visualstudio.com/docs.

2 OMG Systems Modeling Language (SysML) Version 2 [Электронный ресурс]. – Режим доступа : https://www.omg.org/spec/SysML/2.0/.

3 Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма [и др.]. – СПб. : Питер, 2020. – 448 с.

4 Страуструп, Б. Язык программирования C++ / Б. Страуструп. – 4-е изд. – М. : Бином, 2015. – 1136 с.

5 Win32 API [Электронный ресурс] / Microsoft. – Режим доступа : https://learn.microsoft.com/windows/win32/.

6 Жизненный цикл разработки программного обеспечения : лабораторная работа № 1 «Интегрированная среда разработки ПО». – Минск : БГУИР, 2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходные тексты программы

Файл main.cpp

```
#include <windows.h>

#include "controller/ZodiacController.h"
#include "model/ZodiacModel.h"
#include "view/MainWindow.h"

// Точка входа: создание и связывание компонентов MVC.
int WINAPI wWinMain(HINSTANCE instance, HINSTANCE, PWSTR, int showCommand) {
    ZodiacModel model;
    MainWindow view;
    if (!view.create(instance, showCommand)) {
        return 1;
    }
    model.addListener(&view);  // представление подписывается на активную
модель
    ZodiacController controller(model, view);

    MSG msg;
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        if (!IsDialogMessageW(view.handle(), &msg)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
    }
    return static_cast<int>(msg.wParam);
}
```

Файл model/ZodiacSign.h

```
#pragma once

#include <array>
#include <string>

// Знак Зодиака: название, период дат и астрологическая информация.
struct ZodiacSign {
    std::wstring name;
    int startDay, startMonth;  // первый день периода
    int endDay, endMonth;      // последний день периода
    std::wstring element;
    std::wstring planet;
    std::wstring description;
};

// Двенадцать знаков Зодиака в порядке следования по году.
inline const std::array<ZodiacSign, 12>& zodiacSigns() {
    static const std::array<ZodiacSign, 12> signs = {{
        {L"Козерог", 22, 12, 19, 1, L"Земля", L"Сатурн",
         L"Целеустремленный, дисциплинированный и практичный, ценит порядок и
ответственность."},
        {L"Водолей", 20, 1, 18, 2, L"Воздух", L"Уран",
         L"Независимый, изобретательный и дружелюбный, стремится к новому и
необычному."},
```

```
        {L"Рыбы", 19, 2, 20, 3, L"Вода", L"Нептун",
         L"Чуткий, мечтательный и сострадательный, обладает богатым
воображением."},
        {L"Овен", 21, 3, 19, 4, L"Огонь", L"Марс",
         L"Энергичный, смелый и решительный, любит быть первым и брать
инициативу."},
        {L"Телец", 20, 4, 20, 5, L"Земля", L"Венера",
         L"Надежный, терпеливый и упорный, ценит комфорт и стабильность."},
        {L"Близнецы", 21, 5, 20, 6, L"Воздух", L"Меркурий",
         L"Общительный, любознательный и находчивый, легко осваивает
новое."},
        {L"Рак", 21, 6, 22, 7, L"Вода", L"Луна",
         L"Заботливый, эмоциональный и преданный, дорожит семьей и домом."},
        {L"Лев", 23, 7, 22, 8, L"Огонь", L"Солнце",
         L"Щедрый, уверенный в себе и творческий, стремится быть в центре
внимания."},
        {L"Дева", 23, 8, 22, 9, L"Земля", L"Меркурий",
         L"Внимательный к деталям, аналитичный и трудолюбивый, любит
точность."},
        {L"Весы", 23, 9, 22, 10, L"Воздух", L"Венера",
         L"Дипломатичный, справедливый и обаятельный, стремится к
гармонии."},
        {L"Скорпион", 23, 10, 21, 11, L"Вода", L"Плутон",
         L"Страстный, проницательный и настойчивый, обладает сильной
волей."},
        {L"Стрелец", 22, 11, 21, 12, L"Огонь", L"Юпитер",
         L"Оптимистичный, свободолюбивый и открытый, любит путешествия и
знания."},
    }};
    return signs;
}
```

Файл model/IModelListener.h

```
#pragma once

class ZodiacModel;

// Интерфейс наблюдателя активной модели. Представление реализует его,
// чтобы получать уведомления об изменении состояния модели.
class IModelListener {
public:
    virtual ~IModelListener() = default;
    virtual void modelChanged(const ZodiacModel& model) = 0;
};
```

Файл model/ZodiacModel.h

```
#pragma once

#include <vector>

#include "IModelListener.h"
#include "ZodiacSign.h"

// Активная модель: хранит дату рождения, определяет знак Зодиака
// и сама уведомляет подписанные представления об изменениях.
class ZodiacModel {
public:
    static constexpr int MIN_YEAR = 1900;
```

```
    void addListener(IModelListener* listener);
    void removeListener(IModelListener* listener);

    // Проверяет дату рождения, определяет знак и оповещает наблюдателей.
    // При недопустимой дате выбрасывает std::invalid_argument.
    void setBirthDate(int day, int month, int year);

    bool isCalculated() const { return calculated_; }
    int day() const { return day_; }
    int month() const { return month_; }
    int year() const { return year_; }
    const ZodiacSign& sign() const { return zodiacSigns()[signIndex_]; }

    static int daysInMonth(int month, int year);

private:
    static int findSign(int day, int month);
    void fireModelChanged();

    std::vector<IModelListener*> listeners_;
    int day_ = 0, month_ = 0, year_ = 0;
    int signIndex_ = 0;
    bool calculated_ = false;
};
```

Файл model/ZodiacModel.cpp

```
#include "ZodiacModel.h"

#include <algorithm>
#include <ctime>
#include <stdexcept>

void ZodiacModel::addListener(IModelListener* listener) {
    listeners_.push_back(listener);
}

void ZodiacModel::removeListener(IModelListener* listener) {
    listeners_.erase(std::remove(listeners_.begin(), listeners_.end(),
listener),
                     listeners_.end());
}

int ZodiacModel::daysInMonth(int month, int year) {
    static const int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31};
    const bool leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    return month == 2 && leap ? 29 : days[month - 1];
}

void ZodiacModel::setBirthDate(int day, int month, int year) {
    const std::time_t now = std::time(nullptr);
    const std::tm* today = std::localtime(&now);
    const int curYear = today->tm_year + 1900, curMonth = today->tm_mon + 1;
    const int curDay = today->tm_mday;

    if (month < 1 || month > 12) {
        throw std::invalid_argument("Месяц должен быть числом от 1 до 12.");
    }
```

```
    if (year < MIN_YEAR || year > curYear) {
        throw std::invalid_argument("Год должен быть в диапазоне от 1900 до
текущего года.");
    }
    if (day < 1 || day > daysInMonth(month, year)) {
        throw std::invalid_argument("В указанном месяце нет такого дня.");
    }
    if (year == curYear && (month > curMonth || (month == curMonth && day >
curDay))) {
        throw std::invalid_argument("Дата рождения не может быть позже
текущей даты.");
    }
    day_ = day;
    month_ = month;
    year_ = year;
    signIndex_ = findSign(day, month);
    calculated_ = true;
    fireModelChanged();
}

int ZodiacModel::findSign(int day, int month) {
    const auto& signs = zodiacSigns();
    for (int i = 0; i < static_cast<int>(signs.size()); ++i) {
        const ZodiacSign& s = signs[i];
        if ((month == s.startMonth && day >= s.startDay) ||
            (month == s.endMonth && day <= s.endDay)) {
            return i;
        }
    }
    return 0;
}

void ZodiacModel::fireModelChanged() {
    for (IModelListener* listener : listeners_) {
        listener->modelChanged(*this);
    }
}
```

Файл view/Ui.h

```
#pragma once

#include <windows.h>

#include <string>

// Вспомогательные функции создания элементов управления WinAPI.
namespace ui {

inline HFONT font() {
    static HFONT f = CreateFontW(-15, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE, DEFAULT_CHARSET,
                                 OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
CLEARTYPE_QUALITY,
                                 DEFAULT_PITCH, L"Segoe UI");
    return f;
}

inline HFONT boldFont() {
    static HFONT f = CreateFontW(-15, 0, 0, 0, FW_BOLD, FALSE, FALSE, FALSE,
DEFAULT_CHARSET,
```

```
                                    OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
CLEARTYPE_QUALITY,
                                    DEFAULT_PITCH, L"Segoe UI");
    return f;
}

inline HWND create(HWND parent, const wchar_t* cls, const wchar_t* text,
DWORD style, int x, int y,
                   int w, int h, int id = 0, DWORD exStyle = 0) {
    HWND hwnd = CreateWindowExW(exStyle, cls, text, WS_CHILD | WS_VISIBLE |
style, x, y, w, h,
                                parent,
reinterpret_cast<HMENU>(static_cast<INT_PTR>(id)),
                                GetModuleHandleW(nullptr), nullptr);
    SendMessageW(hwnd, WM_SETFONT, reinterpret_cast<WPARAM>(font()), TRUE);
    return hwnd;
}

inline HWND label(HWND parent, const wchar_t* text, int x, int y, int w, int
h) {
    return create(parent, L"STATIC", text, SS_LEFT, x, y, w, h);
}

inline HWND edit(HWND parent, int x, int y, int w, int h) {
    return create(parent, L"EDIT", L"", WS_TABSTOP | ES_AUTOHSCROLL, x, y, w,
h, 0,
                  WS_EX_CLIENTEDGE);
}

inline HWND combo(HWND parent, int x, int y, int w, int h) {
    return create(parent, L"COMBOBOX", L"", WS_TABSTOP | CBS_DROPDOWNLIST, x,
y, w, h);
}

inline HWND button(HWND parent, const wchar_t* text, int id, int x, int y,
int w, int h,
                   bool isDefault) {
    return create(parent, L"BUTTON", text, WS_TABSTOP | (isDefault ?
BS_DEFPUSHBUTTON : 0), x, y,
                  w, h, id);
}

inline std::wstring text(HWND hwnd) {
    std::wstring s(GetWindowTextLengthW(hwnd) + 1, L'\0');
    GetWindowTextW(hwnd, s.data(), static_cast<int>(s.size()));
    s.resize(s.size() - 1);
    return s;
}

}  // namespace ui
```

Файл view/InputDialog.h

```
#pragma once

#include <windows.h>

#include <optional>
#include <string>
```

```
// Дата рождения, введенная пользователем. Хранится как текст, чтобы
// восстановить в окне даже ошибочный ввод.
struct InputData {
    std::wstring dayText;
    std::wstring monthText;
    std::wstring yearText;
};

// Модальное окно ввода даты рождения (стандартный элемент выбора даты
// не используется: день, месяц и год вводятся в текстовые поля).
class InputDialog {
public:
    // Показывает окно и возвращает введенные данные либо пустое значение,
    // если ввод отменен.
    static std::optional<InputData> show(HWND owner, const
std::optional<InputData>& initial);

private:
    InputDialog(HWND owner, const std::optional<InputData>& initial);
    static LRESULT CALLBACK windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam);
    LRESULT handleMessage(UINT msg, WPARAM wParam, LPARAM lParam);
    void createControls();
    void accept();

    HWND owner_;
    HWND hwnd_ = nullptr;
    HWND dayEdit_ = nullptr;
    HWND monthEdit_ = nullptr;
    HWND yearEdit_ = nullptr;
    std::optional<InputData> initial_;
    std::optional<InputData> result_;
    bool done_ = false;
};
```

Файл view/InputDialog.cpp

```
#include "InputDialog.h"

#include "Ui.h"

namespace {
constexpr wchar_t CLASS_NAME[] = L"ZodiacInputDialog";
constexpr int ID_OK = 1;
constexpr int ID_CANCEL = 2;
}  // namespace

std::optional<InputData> InputDialog::show(HWND owner, const
std::optional<InputData>& initial) {
    static bool registered = false;
    if (!registered) {
        WNDCLASSW wc{};
        wc.lpfnWndProc = windowProc;
        wc.hInstance = GetModuleHandleW(nullptr);
        wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
        wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_BTNFACE + 1);
        wc.lpszClassName = CLASS_NAME;
        RegisterClassW(&wc);
        registered = true;
    }
```

```
    InputDialog dialog(owner, initial);
    RECT ownerRect;
    GetWindowRect(owner, &ownerRect);
    const int width = 380, height = 230;
    const int x = ownerRect.left + (ownerRect.right - ownerRect.left - width)
/ 2;
    const int y = ownerRect.top + (ownerRect.bottom - ownerRect.top - height)
/ 2;
    CreateWindowExW(WS_EX_DLGMODALFRAME, CLASS_NAME, L"Дата рождения",
                    WS_POPUP | WS_CAPTION | WS_SYSMENU | WS_VISIBLE, x, y,
width, height,
                    owner, nullptr, GetModuleHandleW(nullptr), &dialog);

    // Модальность: главное окно недоступно, пока открыто окно ввода.
    EnableWindow(owner, FALSE);
    MSG msg;
    while (!dialog.done_ && GetMessageW(&msg, nullptr, 0, 0)) {
        if (!IsDialogMessageW(dialog.hwnd_, &msg)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
    }
    EnableWindow(owner, TRUE);
    SetForegroundWindow(owner);
    DestroyWindow(dialog.hwnd_);
    return dialog.result_;
}

InputDialog::InputDialog(HWND owner, const std::optional<InputData>& initial)
    : owner_(owner), initial_(initial) {}

LRESULT CALLBACK InputDialog::windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam) {
    InputDialog* self = nullptr;
    if (msg == WM_NCCREATE) {
        self =
static_cast<InputDialog*>(reinterpret_cast<CREATESTRUCTW*>(lParam)-
>lpCreateParams);
        self->hwnd_ = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA,
reinterpret_cast<LONG_PTR>(self));
    } else {
        self = reinterpret_cast<InputDialog*>(GetWindowLongPtrW(hwnd,
GWLP_USERDATA));
    }
    return self ? self->handleMessage(msg, wParam, lParam) :
DefWindowProcW(hwnd, msg, wParam, lParam);
}

LRESULT InputDialog::handleMessage(UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:
        createControls();
        return 0;
    case WM_COMMAND:
        if (LOWORD(wParam) == ID_OK || LOWORD(wParam) == IDOK) {
            accept();
        } else if (LOWORD(wParam) == ID_CANCEL || LOWORD(wParam) == IDCANCEL)
{
            done_ = true;  // ввод отменен
        }
```

```
        return 0;
    case WM_CLOSE:
        done_ = true;
        return 0;
    }
    return DefWindowProcW(hwnd_, msg, wParam, lParam);
}

void InputDialog::createControls() {
    ui::label(hwnd_, L"День (1–31):", 16, 20, 180, 24);
    dayEdit_ = ui::edit(hwnd_, 200, 18, 150, 26);
    ui::label(hwnd_, L"Месяц (1–12):", 16, 58, 180, 24);
    monthEdit_ = ui::edit(hwnd_, 200, 56, 150, 26);
    ui::label(hwnd_, L"Год (например, 2005):", 16, 96, 180, 24);
    yearEdit_ = ui::edit(hwnd_, 200, 94, 150, 26);
    ui::button(hwnd_, L"OK", ID_OK, 170, 142, 85, 28, true);
    ui::button(hwnd_, L"Отмена", ID_CANCEL, 265, 142, 85, 28, false);

    // Восстановление последних введенных данных.
    if (initial_) {
        SetWindowTextW(dayEdit_, initial_->dayText.c_str());
        SetWindowTextW(monthEdit_, initial_->monthText.c_str());
        SetWindowTextW(yearEdit_, initial_->yearText.c_str());
    }
    SendMessageW(dayEdit_, EM_SETSEL, 0, -1);
    SetFocus(dayEdit_);
}

void InputDialog::accept() {
    result_ = InputData{ui::text(dayEdit_), ui::text(monthEdit_),
ui::text(yearEdit_)};
    done_ = true;
}
```

Файл view/MainWindow.h

```
#pragma once

#include <windows.h>

#include <functional>
#include <optional>
#include <string>

#include "../model/IModelListener.h"
#include "InputDialog.h"

// Главная форма. Является наблюдателем активной модели: при изменении
// модели сама считывает из нее данные и обновляет метки.
class MainWindow : public IModelListener {
public:
    bool create(HINSTANCE instance, int showCommand);
    HWND handle() const { return hwnd_; }

    void setInputAction(std::function<void()> action) { inputAction_ =
std::move(action); }
    const std::optional<InputData>& lastInput() const { return lastInput_; }
    void setLastInput(const InputData& input) { lastInput_ = input; }
    void showError(const std::wstring& message) const;
```

```
    void modelChanged(const ZodiacModel& model) override;

private:
    static LRESULT CALLBACK windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam);
    LRESULT handleMessage(UINT msg, WPARAM wParam, LPARAM lParam);
    void createControls();

    HWND hwnd_ = nullptr;
    HWND inputButton_ = nullptr;
    HWND dateLabel_ = nullptr;
    HWND signLabel_ = nullptr;
    HWND periodLabel_ = nullptr;
    HWND elementLabel_ = nullptr;
    HWND planetLabel_ = nullptr;
    HWND descriptionLabel_ = nullptr;
    std::function<void()> inputAction_;
    // Последние введенные данные хранятся в главной форме.
    std::optional<InputData> lastInput_;
};
```

Файл view/MainWindow.cpp

```
#include "MainWindow.h"

#include <cstdio>

#include "../model/ZodiacModel.h"
#include "Ui.h"

namespace {
constexpr wchar_t CLASS_NAME[] = L"ZodiacMainWindow";
constexpr int ID_INPUT = 100;

const wchar_t* MONTHS[] = {L"января", L"февраля", L"марта", L"апреля",
L"мая", L"июня",
                           L"июля", L"августа", L"сентября", L"октября",
L"ноября", L"декабря"};

std::wstring dayMonth(int day, int month) {
    return std::to_wstring(day) + L" " + MONTHS[month - 1];
}
}  // namespace

bool MainWindow::create(HINSTANCE instance, int showCommand) {
    WNDCLASSW wc{};
    wc.lpfnWndProc = windowProc;
    wc.hInstance = instance;
    wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_BTNFACE + 1);
    wc.lpszClassName = CLASS_NAME;
    RegisterClassW(&wc);

    hwnd_ = CreateWindowExW(0, CLASS_NAME, L"Астролог",
                            WS_OVERLAPPED | WS_CAPTION | WS_SYSMENU |
WS_MINIMIZEBOX,
                            CW_USEDEFAULT, CW_USEDEFAULT, 560, 360, nullptr,
nullptr, instance,
                            this);
    if (!hwnd_) return false;
```

```
    ShowWindow(hwnd_, showCommand);
    UpdateWindow(hwnd_);
    return true;
}

LRESULT CALLBACK MainWindow::windowProc(HWND hwnd, UINT msg, WPARAM wParam,
LPARAM lParam) {
    MainWindow* self = nullptr;
    if (msg == WM_NCCREATE) {
        self =
static_cast<MainWindow*>(reinterpret_cast<CREATESTRUCTW*>(lParam)-
>lpCreateParams);
        self->hwnd_ = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA,
reinterpret_cast<LONG_PTR>(self));
    } else {
        self = reinterpret_cast<MainWindow*>(GetWindowLongPtrW(hwnd,
GWLP_USERDATA));
    }
    return self ? self->handleMessage(msg, wParam, lParam) :
DefWindowProcW(hwnd, msg, wParam, lParam);
}

LRESULT MainWindow::handleMessage(UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:
        createControls();
        return 0;
    case WM_SETFOCUS:
        SetFocus(inputButton_);  // кнопка нажимается и с клавиатуры
        return 0;
    case WM_COMMAND:
        if ((LOWORD(wParam) == ID_INPUT || LOWORD(wParam) == IDOK) &&
inputAction_) {
            inputAction_();  // действие пользователя передается контроллеру
        }
        return 0;
    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProcW(hwnd_, msg, wParam, lParam);
}

void MainWindow::createControls() {
    inputButton_ = ui::button(hwnd_, L"Узнать свой знак Зодиака", ID_INPUT,
16, 16, 512, 32, true);
    const wchar_t* captions[] = {L"Дата рождения:", L"Знак Зодиака:",
L"Период:", L"Стихия:",
                                 L"Планета-покровитель:",
L"Характеристика:"};
    HWND* values[] = {&dateLabel_, &signLabel_, &periodLabel_,
&elementLabel_, &planetLabel_,
                      &descriptionLabel_};
    for (int i = 0; i < 6; ++i) {
        const int y = 66 + i * 34;
        ui::label(hwnd_, captions[i], 16, y, 170, 24);
        *values[i] = ui::label(hwnd_, i == 0 ? L"не введена" : L"—", 190, y,
338, i == 5 ? 60 : 24);
    }
    SendMessageW(signLabel_, WM_SETFONT,
reinterpret_cast<WPARAM>(ui::boldFont()), TRUE);
}
```

```
void MainWindow::showError(const std::wstring& message) const {
    MessageBoxW(hwnd_, message.c_str(), L"Некорректные данные", MB_OK |
MB_ICONERROR);
}

void MainWindow::modelChanged(const ZodiacModel& model) {
    if (!model.isCalculated()) return;
    wchar_t date[32];
    swprintf(date, 32, L"%02d.%02d.%04d", model.day(), model.month(),
model.year());
    const ZodiacSign& s = model.sign();
    SetWindowTextW(dateLabel_, date);
    SetWindowTextW(signLabel_, s.name.c_str());
    SetWindowTextW(periodLabel_, (L"с " + dayMonth(s.startDay, s.startMonth)
+ L" по " +
                                  dayMonth(s.endDay, s.endMonth)).c_str());
    SetWindowTextW(elementLabel_, s.element.c_str());
    SetWindowTextW(planetLabel_, s.planet.c_str());
    SetWindowTextW(descriptionLabel_, s.description.c_str());
}
```

Файл controller/ZodiacController.h

```
#pragma once

#include <string>

class ZodiacModel;
class MainWindow;

// Контроллер: обрабатывает действия пользователя, проверяет введенные
// данные и изменяет модель. Представление контроллер не обновляет –
// при активной модели это делает сама модель через механизм уведомлений.
class ZodiacController {
public:
    ZodiacController(ZodiacModel& model, MainWindow& view);

    // Преобразует текст в целое число; при ошибке выбрасывает
std::invalid_argument.
    static int parseNumber(const std::wstring& text, const char* field);

private:
    void onInputData();  // обработчик кнопки «Узнать свой знак Зодиака»

    ZodiacModel& model_;
    MainWindow& view_;
};
```

Файл controller/ZodiacController.cpp

```
#include "ZodiacController.h"

#include <windows.h>

#include <cwchar>
#include <stdexcept>
#include <string>
```

```
#include "../model/ZodiacModel.h"
#include "../view/InputDialog.h"
#include "../view/MainWindow.h"

namespace {
std::wstring fromUtf8(const char* text) {
    const int size = MultiByteToWideChar(CP_UTF8, 0, text, -1, nullptr, 0);
    std::wstring result(size, L'\0');
    MultiByteToWideChar(CP_UTF8, 0, text, -1, result.data(), size);
    result.resize(size - 1);
    return result;
}
}  // namespace

ZodiacController::ZodiacController(ZodiacModel& model, MainWindow& view)
    : model_(model), view_(view) {
    view_.setInputAction([this] { onInputData(); });
}

void ZodiacController::onInputData() {
    std::optional<InputData> input = view_.lastInput();
    while (true) {
        input = InputDialog::show(view_.handle(), input);
        if (!input) {
            return;  // пользователь отменил ввод
        }
        view_.setLastInput(*input);  // запоминаем последние данные в главной
форме
        try {
            const int day = parseNumber(input->dayText, "День");
            const int month = parseNumber(input->monthText, "Месяц");
            const int year = parseNumber(input->yearText, "Год");
            model_.setBirthDate(day, month, year);
            return;
        } catch (const std::invalid_argument& ex) {
            view_.showError(fromUtf8(ex.what()));
        }
    }
}

int ZodiacController::parseNumber(const std::wstring& text, const char*
field) {
    const size_t first = text.find_first_not_of(L" \t");
    if (first == std::wstring::npos) {
        throw std::invalid_argument(std::string("Поле «") + field + "» не
заполнено.");
    }
    const wchar_t* begin = text.c_str() + first;
    wchar_t* end = nullptr;
    const long value = std::wcstol(begin, &end, 10);
    while (end && (*end == L' ' || *end == L'\t')) ++end;
    if (end == begin || *end != L'\0') {
        throw std::invalid_argument(std::string("Поле «") + field + "» должно
содержать целое число.");
    }
    return static_cast<int>(value);
}
```